

Dust 复现

Dust (Dahal 等, 2026) 不做反向传播就能训练 transformer 语言模型：它在每个 token 的激活上加噪声，再把随之产生的 loss 变化转成梯度估计。论文称它在预训练上与反向传播 (backpropagation, 下称 backprop) 相当。我们跑了官方 release, 补上了它缺少的 AdamW baseline, 并在 backprop 已经训练过的模型上测试了 Dust。

要点

- 官方 release 能复现论文。它用随机梯度下降 (stochastic gradient descent, SGD) 的 Dust 运行, 以及我们重建的 Adam 版 Dust, test loss 与论文数值的差都在 0.05 以内 (表 1)。
- 论文的主要对比里, backprop 用的是恒定学习率的 SGD。换成 AdamW 后, backprop 在 10M token 上好了约 1.0 nat, 用 SGD 的 Dust 没有一格能接近它 (表 2)。
- 论文只在 1M token 上报告了 Adam 版 Dust; 在那里最大 population 的 Dust 与 backprop AdamW 持平。到 10M token, 它在前一百步跟得上 backprop, 之后持续落后, population 为 16,384 时最终高出约 0.5 nat (图 2)。调学习率或 β_1 补不上这个差距 (表 4)。
- 从 backprop 已经训练好的模型出发, Dust 仍能继续降低 loss, 但前提是学习率低于从头训练时调出的值。population 为 16,384 时, 它在我们的两个 checkpoint 上分别达到 backprop 同期降幅的 73% 和 85% (表 6)。没有迹象表明它能找到 backprop 找不到的方向。
- 代价很大。1M token 上 population 为 16,384 的运行, 所用的图形处理器 (graphics processing unit, GPU) 时间约为 backprop 的 600 倍 (表 7)。
- 本次复现没有覆盖的内容 (包括 16,384 以上的 population) 见下面的范围说明和文末的局限。

范围与免责说明

这是一次独立复现, 与论文作者无关。它只覆盖一个模型尺寸 (37.7M 参数)、最多 10M token 的从头训练、最大 16,384 的 population; 除非表中另有说明, 每个 Dust 运行只有一个 seed。Adam 版 Dust 是我们根据论文附录重建的。本报告由 Claude Opus 5.5 根据运行记录起草, 如有错误由我们负责。

我们没有检验更大的 population 能否补上与 AdamW 的差距。在 10M token 上, population 从 4,096 增到 16,384 (四倍), 与 backprop AdamW 的差距从 0.62 nat 缩小到 0.50 nat, 算力是四倍 (表 2)。如果之后每次四倍都能带来同样的改善, 补上差距还需要约 4 次四倍, 即百万量级的 population。这是只用两个点、各一个 seed 做的外推, 而且更早的几次四倍改善更大, 所以只能当作量级参考。

跑了什么

对一个在 token t 处输出 y_t 的线性层，Dust 加上噪声 σa_t ($a_t \sim \mathcal{N}(0, I)$)，重跑前向，并用该噪声带来的 loss 下降量 r_t 给它打分。对 K 次独立抽样 (draw) 取平均，

$$\hat{g}_t = -\frac{1}{K\sigma} \sum_{i=1}^K r_t^{(i)} a_t^{(i)}$$

就得到该层输出处的误差估计，它与该层缓存输入的外积即权重梯度。 K 称为 population。注意力的 query、key 和 value 不直接用 loss 打分，而是用注意力输出的误差估计打分。想看推导的读者可以展开下面一节。

推导：为什么“噪声 $\times$ loss 变化”能估出梯度

要估计的量。对 $y_t = Wx_t$ ，backprop 给出 $\partial L / \partial W = \sum_t \delta_t x_t^\top$ ，其中 $\delta_t = \partial L / \partial y_t$ 。输入 x_t 在前向时已知，只有 δ_t 需要反向传播。Dust 不做反向传播来估计 δ_t 。

估计器。噪声加在该层的输出上，而不是权重上：每个 token s 处 $y_s \rightarrow y_s + \sigma a_s$ ， W 不变。把 loss 对这些输出做一阶泰勒展开，扰动后的 loss 是 $L + \sigma \sum_s \delta_s^\top a_s$ ，其中 $\delta_s = \partial L / \partial y_s$ 就是上面定义的量在 token s 处的取值；下标 s 遍历所有 token， t 是当前要估计的那个。于是打分 $r \approx -\sigma \sum_s \delta_s^\top a_s$ 。把它乘上某个 token 的噪声再取期望，

$$-\frac{1}{\sigma} \mathbb{E}[r a_t] = \mathbb{E}[a_t a_t^\top] \delta_t + \sum_{s \neq t} \mathbb{E}[a_t a_s^\top] \delta_s = \delta_t,$$

因为 $\mathbb{E}[a_t a_t^\top] = I$ ，且不同 token 的噪声相互独立、均值为零。把期望换成 K 次 draw 的平均，就得到 $\hat{g}_t$ 。

逐 token 打分。总 loss 的变化里还包含其他所有 token 的噪声造成的部分，这部分打分和 a_t 毫无关系。对很多次 draw 取平均时它趋于零，所以不会让估计有偏；但在单次 draw 里，它是一个很大的随机数乘在 a_t 上：序列长为 T 时，它的方差大约是信号的 T 倍，也就是需要约 T 倍的 draw 才能平均掉。所以 Dust 不用总 loss 的变化，而是用各个位置上的 loss 下降量 c_s 来打分。记 ℓ_s 为位置 s 处预测下一个 token 的交叉熵， $\ell_s^{(i)}$ 为第 i 次 draw 下的取值。对同一次批量前向里一起评估的 n 次 draw， $c_s^{(i)} = \frac{1}{n} \sum_{j=1}^n \ell_s^{(j)} - \ell_s^{(i)}$ ，即这次 draw 下位置 s 的 loss 比平均 draw 低多少。token t 处噪声的打分则是

$$r_t = \sum_{s \geq t} \gamma^{s-t} c_s.$$

在因果注意力下， t 处的扰动改变不了更早 token 的 loss，所以丢掉它们只会减小方差。衰减 γ 在偏差和方差之间折中。 $\gamma = 1$ 时，估计在一阶近似下是 backprop 的 δ_t 的无偏估计。 $\gamma = 0$ 时只保留本 token 的 loss，估计的是 $\partial \ell_t / \partial y_t$ ，漏掉了后面通过注意力读取这个激活的 token。论文对用

loss 打分的层取 $\gamma = 0$ ，对 key 和 value 取约 0.98。 c_s 用同批 draw 的均值而不是未扰动的 loss 作基线，是因为噪声会让 loss 整体偏移一个二阶小量（通常是升高），每次 draw 都差不多；这个公共偏移乘在 a_t 上就是纯噪声。只要基线与 a_t 无关，由于 $\mathbb{E}[a_t] = 0$ ，期望不变。这里的均值包含了本次 draw 自己，所以估计会按已知系数 $(n - 1)/n$ 缩小，方向不变。release 里 n 随 population 取 2 到 16。

为什么 population 必须很大。单次 draw 的估计 $a(a^\top \delta)$ 均值正确，但 a 有 d 个维度，只有沿 δ 的那一个带信号。 K 次 draw 之后，它与真实梯度的余弦大致是 $1/\sqrt{1 + c/K}$ ，其中 c 与 d 同量级，再加上其他被扰动的 token 和层带来的干扰。论文用 $c_{\max}/\sqrt{1 + c/K}$ 拟合它测到的余弦。

虚拟 population。在 token t 的输出上加 σa_t ，等价于只在这个 token 上使用权重 $W + \sigma a_t x_t^\top / \|x_t\|^2$ ，即一个秩一的权重扰动。每个 token 有自己的 a_t ，所以一次前向评估的权重扰动数等于 batch 里的 token 数，而权重空间的进化策略一次只评估一个。

注意力内部。query、key、value 上的噪声对 loss 的影响太小，读不出来。Dust 先按上面的办法估出注意力输出处的误差 $\hat{g}_s^o$ ，再只重算这一层在噪声下的注意力，用 $c_s = -\langle \hat{g}_s^o, \Delta o_s \rangle$ 给输出的变化 Δo_s 打分，作为 loss 下降量的线性替代。

以下是我们的解读，不是论文的说法： $\gamma = 0$ 时，估计漏掉了一个激活对后续 token 的影响，所以它逼近的是截断后的梯度，而不是 backprop 的梯度。这可能是多数层拟合出的上限 $c_{\max}$ 低于 1 的原因之一。另外，随着训练推进，估计的噪声相对信号越来越大，这与下文看到的现象相符：与 backprop 的差距随步数扩大，训练过的 checkpoint 需要更小的学习率。

论文	Dahal、Mandal、Gülbahar 与 Vegesna, Dust: Pretraining Transformers Without Backpropagation , Q Labs, 2026 年 10 月。
代码	官方 release , commit 2fdb01b; 所有 SGD 结果都用未改动的原代码。它是最小实现：每次 draw 重跑整个前向，而论文只重跑被扰动层之后的部分。
模型	仅解码器的 transformer，8 层、宽度 512、37.7M 参数：旋转位置编码、归一化的 query 与 key、平方 ReLU 的前馈层、value embedding，以及不共享权重的输出 head。
数据	FineWeb，4,096 词表的字节对编码（byte-pair encoding, BPE），每个 batch 为 8 条 2,048 token 的序列：1M token 是 61 步，10M 是 610 步。验证集和测试集各 544 条留出序列。
指标	验证 loss 最低的 checkpoint 上的 test loss，与论文一致。
我们补充的	按论文 Adam 设置（附录 F）训练的 backprop AdamW；因 release 只有 SGD 而根据附录 E、F 重建的 Adam 版 Dust；以及从 backprop checkpoint 出发的续训。
seed	backprop 3 个，Dust 每个运行 1 个（42）。

release 覆盖 1M 与 10M token、population 256 到 16,384 的 SGD 运行。它不包含 population 64、100k 与 20M token 的预算、EGGROLL baseline、Adam 运行、模型尺寸实验和梯度余弦测量，所以除了 1M token 的 Adam 以外，这些都没有在这里核对。

release 能复现论文吗？

能。release 覆盖的每一格都落在论文数值上，backprop SGD 也是（表 1、图 1）。Adam 的几行同时检验了我们的重建：附录给了 Adam 下每类层的 draw 总数，但没给它们在各层之间的分配，我们按 SGD 的分配等比缩放。1M token 上论文公布的四个点都对上了。

表 1 test loss 与论文数值的对照。论文数值是 3 个 seed（SGD）或 5 个 seed（Adam）的均值；我们的 Dust 是单个 seed。论文在 10M token 上没有 AdamW 或 Adam 的运行。

运行	我们	论文	差
1M TOKEN（61 步）			
Backprop（SGD），3 个 seed	5.959	5.959	0.000
Backprop（AdamW），3 个 seed	5.364	5.361	0.003
Dust（SGD），256	6.054	6.067	-0.013
Dust（SGD），1k	5.933	5.934	-0.001
Dust（SGD），4k	5.878	5.911	-0.033
Dust（SGD），16k	5.862	5.916	-0.054
Dust（Adam），256	5.843	5.836	0.007
Dust（Adam），1k	5.603	5.594	0.009
Dust（Adam），4k	5.499	5.500	-0.001
Dust（Adam），16k	5.382	5.402	-0.020
10M TOKEN（610 步）			
Backprop（SGD），3 个 seed	4.989	4.989	0.000
Backprop（AdamW），3 个 seed	3.939	-	-
Dust（SGD），256	5.230	5.231	-0.001
Dust（SGD），1k	5.118	5.111	0.007
Dust（SGD），4k	5.070	5.072	-0.002
Dust（SGD），16k	5.047	5.049	-0.002
Dust（Adam），256	5.171	-	-
Dust（Adam），1k	4.854	-	-
Dust（Adam），4k	4.560	-	-
Dust（Adam），16k	4.436	-	-

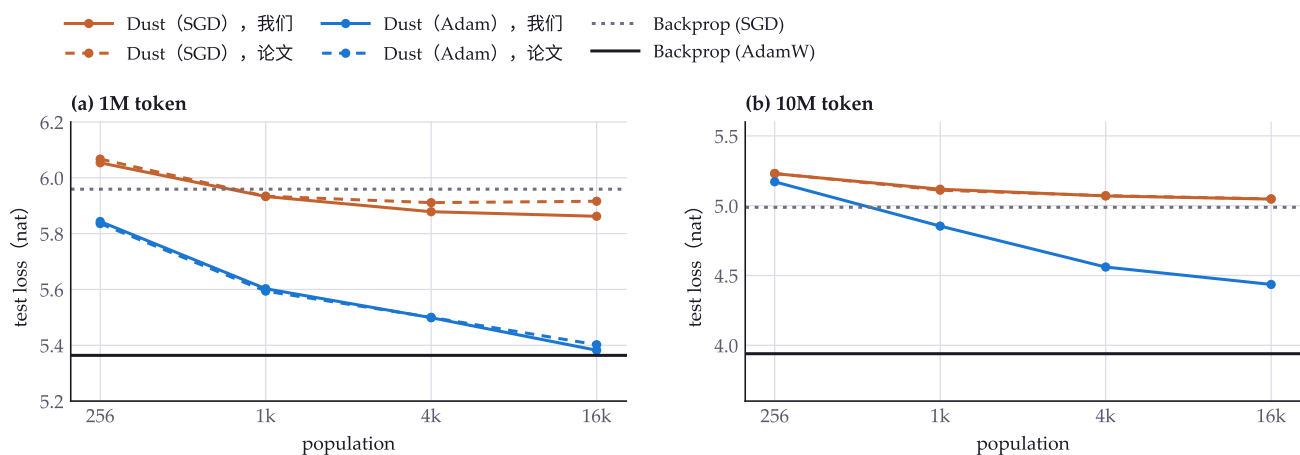


图 1 test loss 随 population 的变化。带点的实线是我们的运行，虚线是论文公布的均值。水平线是 backprop：SGD（点线）与 AdamW（实线）。

与 AdamW 对比

在同一种优化器内，论文的结论成立。用 SGD 时，Dust 在 1M token 上从 population 约一千起就与 backprop 持平，在 10M 上紧随其后。但恒定学习率的 SGD 并不是训练这个模型的好办法。backprop 换成 AdamW 后在两个预算上都好得多，而且它对 SGD 的领先随 token 数增大（表 2）。论文自己的 Adam 实验只做到 1M token，整个训练只有 61 步。

表 2 各方法的 test loss 及其高出 backprop AdamW 的量。backprop 行是 3 个 seed 的均值，Dust 行是单个 seed。

方法	1M: test loss	1M: 高于 AdamW	10M: test loss	10M: 高于 AdamW
Backprop (SGD)	5.959	0.60	4.989	1.05
Backprop (AdamW)	5.364	0.00	3.939	0.00
Dust (SGD), 256	6.054	0.69	5.230	1.29
Dust (SGD), 1k	5.933	0.57	5.118	1.18
Dust (SGD), 4k	5.878	0.51	5.070	1.13
Dust (SGD), 16k	5.862	0.50	5.047	1.11
Dust (Adam), 256	5.843	0.48	5.171	1.23
Dust (Adam), 1k	5.603	0.24	4.854	0.91
Dust (Adam), 4k	5.499	0.13	4.560	0.62
Dust (Adam), 16k	5.382	0.02	4.436	0.50

在 10M token 上，从 population 1k 起 Adam 版 Dust 都优于 SGD 版，并且超过了 backprop SGD。和 backprop AdamW 相比，结论取决于看训练的哪个阶段（图 2）。前一百步里，最大 population 的 Dust 几乎贴着 backprop；此后 population 1k 及以上的差距持续扩大，直到训练结束；population 256 的 Dust

起步慢，差距先缩小再扩大（表 3）。更大的 population 只是推迟了分叉，并没有消除它，这与论文自己的说法一致：要追平 backprop 所需的 population 随 token 数增长。

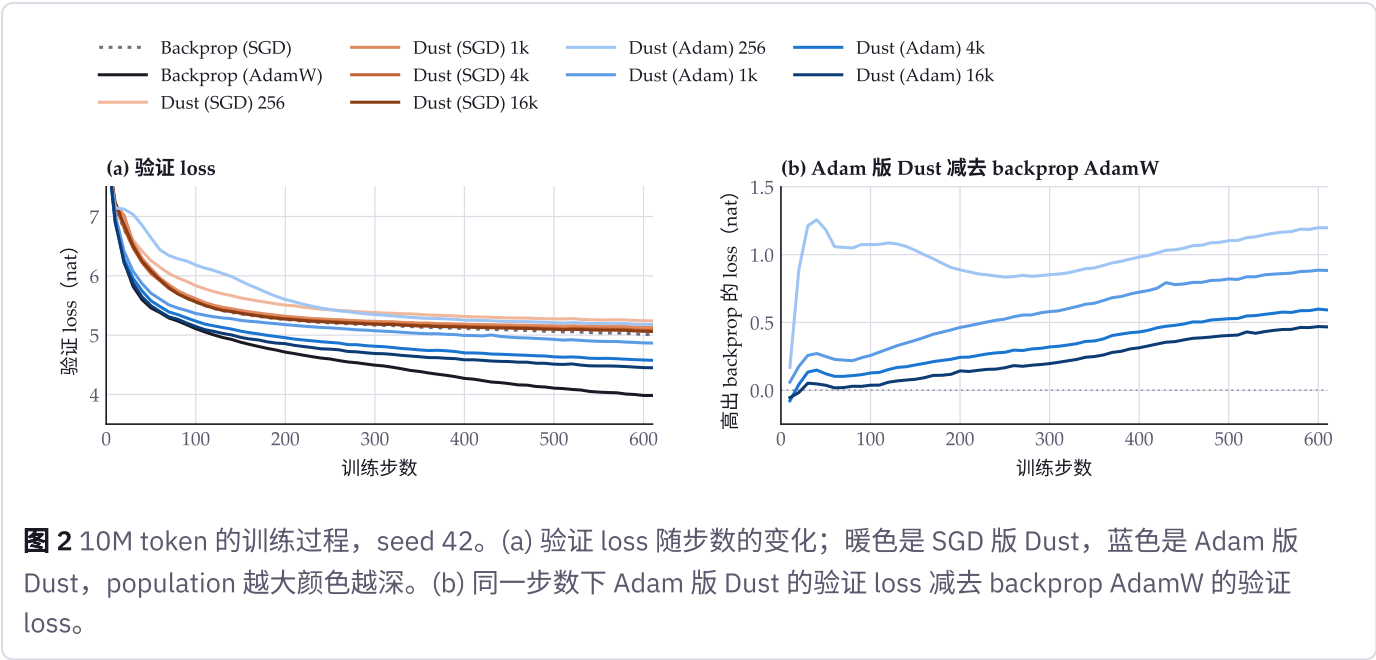


图 2 10M token 的训练过程，seed 42。(a) 验证 loss 随步数的变化；暖色是 SGD 版 Dust，蓝色是 Adam 版 Dust，population 越大颜色越深。(b) 同一步数下 Adam 版 Dust 的验证 loss 减去 backprop AdamW 的验证 loss。

表 3 10M token 上 backprop AdamW 的验证 loss，以及同一步数下 Adam 版 Dust 高出它的量（seed 42）。

步数	100	200	300	400	500	610
Backprop (AdamW)，验证 loss	5.11	4.71	4.49	4.27	4.11	3.98
Dust (Adam)，256	1.07	0.89	0.85	0.98	1.10	1.20
Dust (Adam)，1k	0.26	0.46	0.58	0.72	0.82	0.88
Dust (Adam)，4k	0.13	0.24	0.32	0.43	0.53	0.59
Dust (Adam)，16k	0.04	0.14	0.20	0.31	0.40	0.47

调参能补上差距吗？

在我们试过的两个方向上不能。Dust 的 Adam 设置来自论文 61 步的运行，所以我们在 population 1k 上改了最可能随训练长度变化的两项：学习率和 β_1 。学习率减半略有帮助，其余改动都更差（表 4）。backprop 的学习率在两个预算上都被夹住，论文的取值最好。

表 4 不同优化器设置下的 test loss，seed 42。

设置	test loss
BACKPROP（ADAMW），矩阵学习率；SEED 42	
1M token，学习率 0.001	5.555
1M token，学习率 0.002	5.372
1M token，学习率 0.004	5.453
10M token，学习率 0.0005	4.160
10M token，学习率 0.001	3.987
10M token，学习率 0.002	3.977
10M token，学习率 0.004	4.154
DUST（ADAM），POPULATION 1K，10M TOKEN；SEED 42	
附录设置：学习率 0.0015， β_1 0.7	4.854
学习率减半（0.00075）	4.832
学习率加倍（0.003）	4.987
β_1 0.9	4.976
β_1 0.95	5.003

为了看结果随 seed 的波动有多大，我们把一个 Dust 格子用三个 seed 重复，并与 backprop 的三个 seed 并列（表 5）。

表 5 10M token 上各 seed 的 test loss。短横线表示本页生成时尚未跑完的运行。

方法	Seed 42	Seed 43	Seed 44	已完成的均值
Backprop（AdamW）	3.977	3.901	3.940	3.939
Dust（Adam），4k	4.560	—	—	4.560

我们没有改变 draw 在各层之间的分配、噪声尺度和 credit decay。论文在 10M token 的最大一格上为 SGD 调过这些，称只有更慢的 decay 和把 draw 向注意力一侧倾斜有收益。

从 backprop checkpoint 续训

论文认为 backprop 对 loss 地形的探索不足。若是如此，零阶方法应该能改进 backprop 训练过的模型，这是 You Jiacheng 提出的检验。我们用 backprop AdamW 训了两个 checkpoint，分别用 10M 和 100M token，然后各续训 200 步，所用 token 既不在 checkpoint 的训练数据里，也不在留出集里。除一组外，所有运行都从全新的优化器状态开始，以便 backprop 与 Dust 受到同等对待；剩下那组保留 backprop 的 AdamW 状态，作为不中断训练的参照。

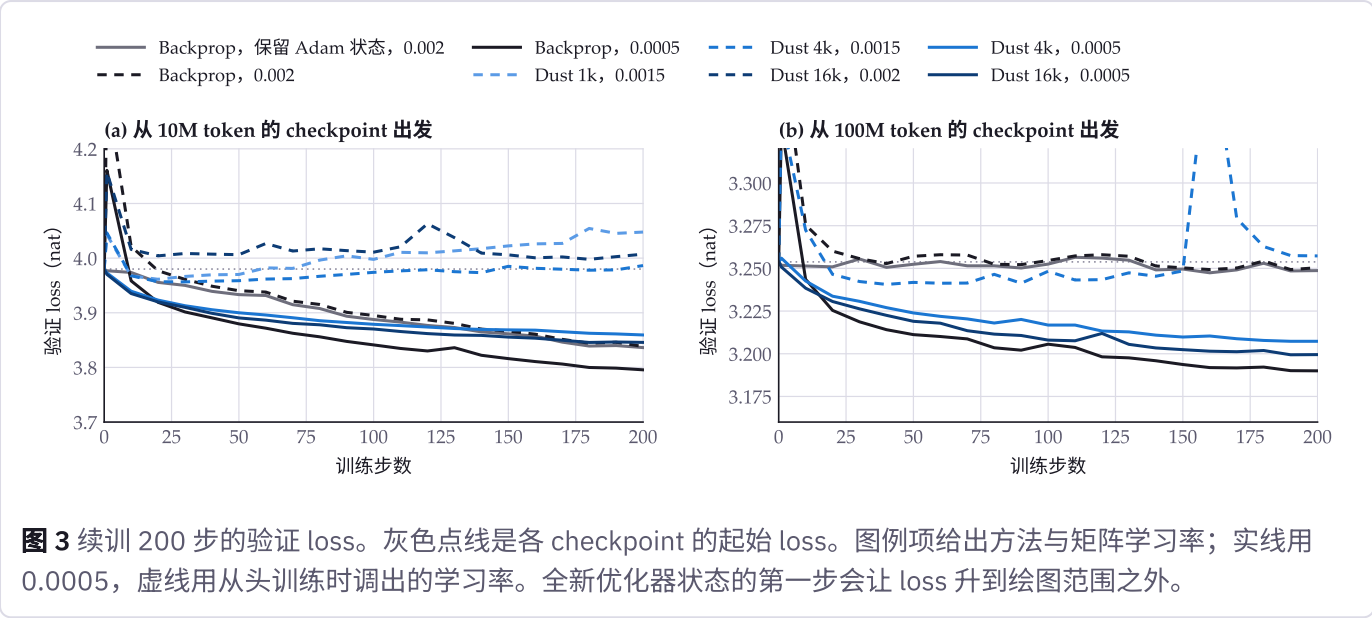


表 6 续训过程中的验证 loss。最后一列是 Dust 到第 200 步时相对起始 loss 的降幅，占同样学习率 0.0005 下 backprop 降幅的比例。Dust 的学习率指矩阵的学习率；embedding 与标量的学习率按附录的比例随之缩放。

方法与矩阵学习率	第 50 步	第 100 步	第 200 步	占 backprop 降幅的比例
从 10M TOKEN 的 CHECKPOINT 出发（验证 LOSS 3.980）				
Backprop，保留 Adam 状态，0.002	3.933	3.888	3.836	—
Backprop，0.002	3.940	3.895	3.838	—
Backprop，0.001	3.891	3.849	3.799	—
Backprop，0.0005	3.880	3.841	3.796	—
Dust 1k，0.0015	3.970	3.998	4.048	—
Dust 4k，0.0015	3.959	3.974	3.986	—
Dust 16k，0.002	4.006	4.011	4.007	—
Dust 4k，0.0005	3.900	3.879	3.859	65%
Dust 16k，0.0005	3.891	3.870	3.846	73%
从 100M TOKEN 的 CHECKPOINT 出发（验证 LOSS 3.254）				
Backprop，保留 Adam 状态，0.002	3.252	3.252	3.249	—
Backprop，0.002	3.257	3.255	3.250	—
Backprop，0.001	3.221	3.215	3.204	—
Backprop，0.0005	3.211	3.206	3.190	—
Backprop，0.00025	3.213	3.206	3.190	—
Dust 4k，0.0015	3.242	3.248	3.257	—
Dust 4k，0.0005	3.224	3.217	3.207	73%
Dust 16k，0.0005	3.219	3.208	3.199	85%

- 用从头训练时调出的学习率，Dust 对两个 checkpoint 都没有改进：每个 population 下，最终 loss 都不低于起点。
- 把学习率降到 0.0005 后，Dust 在两个 checkpoint 上都能稳定降低 loss，population 越大降得越多；但它始终高于同学习率的 backprop（图 3）。
- backprop 同样受益于更低的学习率；在 100M token 的 checkpoint 上，原学习率已经完全不再下降。所以这个 checkpoint 主要检验的是：噪声更大的梯度估计配上小步长还能不能用，而不是 Dust 是否探索得更好。

原提议要求的是 GPT-2 tokenizer 下 loss 低于 3 的 checkpoint。我们的起点是 4,096 词表下的 3.98 和 3.25。不同 tokenizer 的 loss 不能直接比较，而这两个模型肯定比提议中的弱。

代价

Dust 的每一单位 population 都要付出一次前向。用这份 release，最大 population 需要 8 张 GPU 跑数小时，而 backprop 只需 1 张 GPU 跑数秒到数分钟（表 7）。有两点会让比值偏移：计时包含评估，而评估占了 backprop 墙钟时间的大部分，所以比值低估了训练算力的差距；另外最小实现每次 draw 都重跑整个网络，论文自己的实现避免了这一点。

表 7 墙钟开销，含评估。最后一列是相对同一 token 预算下 backprop AdamW 的 GPU 小时倍数。

运行	GPU 数	每步秒数	GPU 小时	相对 backprop
1M TOKEN，61 步				
Backprop (AdamW)	1	0.54	0.009	1
Dust (SGD)，256	1	5.3	0.09	10
Dust (SGD)，1k	1	16.9	0.29	31
Dust (SGD)，4k	4	16.8	1.14	125
Dust (SGD)，16k	8	32.1	4.35	478
Dust (Adam)，256	1	6.7	0.11	12
Dust (Adam)，1k	1	22.0	0.37	41
Dust (Adam)，4k	4	21.8	1.48	162
Dust (Adam)，16k	8	42.2	5.73	629
10M TOKEN，610 步				
Backprop (AdamW)	1	0.14	0.024	1
Dust (SGD)，256	1	4.8	0.82	34
Dust (SGD)，1k	1	16.5	2.79	114
Dust (SGD)，4k	4	16.3	11.03	453
Dust (SGD)，16k	8	31.5	42.69	1,752
Dust (Adam)，256	1	6.2	1.04	43
Dust (Adam)，1k	1	21.5	3.64	149
Dust (Adam)，4k	4	21.3	14.43	592
Dust (Adam)，16k	8	41.6	56.43	2,316

外部批评

对论文的三条公开回应（引自 [Digg 的讨论汇总](#)），以及我们的运行对每一条的说明（表 8）。

表 8 批评与本次复现的对应结果。

谁	观点	我们的运行显示
Sadhika Malladi	FineWeb 验证 loss 约 5 算不上有意义的预训练量；图的横轴应该用步数或 GPU 小时，而不是 token 数。	成立。同样的 10M token，backprop AdamW 能降到 4 以下（表 2）；GPU 小时的比值见 表 7。
You Jiacheng	如果 backprop 缺乏搜索，就拿 backprop 训好的 checkpoint 用零阶方法续训，看它是否有效。	在比提议更弱的 checkpoint 上做了检验。降低学习率后 Dust 能继续改进它们，但始终落后于 backprop（表 6）。
Kaiyue Wen	问能否直接用梯度下降来优化激活。	这里没有检验。

本次复现的局限

- 除 表 5 中的那一格外，每个 Dust 运行只有一个 seed。backprop 的三个 seed 在 1M token 上相差约 0.06，在 10M 上约 0.08，所以 Dust 各格之间这个量级的差别无法分辨。
- Adam 版 Dust 是重建的。draw 在各层之间的分配是我们的假设；它在 1M token 上与论文公布的四个点一致，在 10M 上没有可对照的公布数值。
- 10M token 上 Dust 的 Adam 设置只在一个 population 上沿两个方向调过。
- 这里的最大 population 是 16,384。论文在 1M token 的 Adam 实验做到 65,536，而它"最终低于 backprop"的说法依赖更远的外推。
- 续训只有 200 步、两个 checkpoint，Dust 只试了一个低于默认值的学习率。

参考文献

- Dahal, Mandal, Gülbahar and Vegesna (2026). [Dust: Pretraining Transformers Without Backpropagation. Code.](#)
- Malladi et al. (2023). [Fine-Tuning Language Models with Just Forward Passes.](#)
- Sarkar et al. (2025). [Evolution Strategies at the Hyperscale.](#)
- Digg (2026). [Dust debuts as a zeroth-order approach to transformer pretraining.](#)

代码、每个运行的记录和本报告的 PDF 都在 github.com/huskydoge/dust，它是官方 release 的 fork。它新增了 `dust_adam.py`、`dust_cpt.py`、`baselines/backprop_adam.py` 和 `prepare_cpt_data.py`；官方的 `dust.py` 与 `baselines/backprop.py` 未改动。